

MICROPROCESOARE SI LIMBAJE DE ASAMBLARE

Calculatoare 3 2025-2026

Curs 10 Sapt 10 2 decembrie 2025 16:00-18:00

serban@upit.ro

Modul de notare:

Examen:	50%	
Laborator:	30%	A2
Test (Midterm):	20%	A1

Planificarea activitatilor:

Laborator:

- saptamanal, conform orarului;
- sapt. 13, 14 refaceri laboratoare;

Curs:

- saptamanal, conform orarului;

Test (Midterm): in saptamana a 11-a (**16 decembrie 2025**)

OBS. Este nevoie de nota minima 5 la toate activitatile (semestru – A1, A2 si sesiune – Examen!!! pentru promovare)

Continut

Introducere

Evolutia microprocesoarelor

Criterii de clasificare ale microprocesoarelor

Structura microprocesoarelor

Mecanisme specifice Microprocesoarelor

Microprocesoare pe 8 biti

Familia de microprocesoare x86

Familia de procesoare ARM

Bibliografie

The Design of a Microprocessor – Wilhelm G. Spruth – Springer

Microprocessor Design – Grant McFarland – McGraw-Hill – 2006

Microprocessor Interface Design – J.D. Nicoud – Chapman & Hall

Microprocessors and Microcomputers Hardware and Software - Ronald J. Tocci – Pearson

Intel Microprocessors – Barry Brey

Z-80 Microprocessor Architecture, Interfacing, Programming and Design - Ramesh Gaonkar

Introduction to 80X86 Assembly Language and Computer Architecture - Richard C. Detmer - Jones & Bartlett Pub (2001)

X86 Assembly Language and C Fundamentals – Joseph Cavanagh – CRC Press

ARM Assembly Language Fundamentals and Techniques, Second Edition - William Hohl

Fundamentals of Embedded Software Where C and Assembly Meet – Daniel Wesley Lewis

În sinteza, afectarea conținutului registrului PC poate fi făcută în următoarele moduri:

- ✓ La inițializarea procesorului (reset) când PC-ul se încarcă cu adresa de memorie de la care este scris programul de lucru al acestuia; respectiva adresă diferă de la procesor la procesor, iar zona corespunzătoare de memorie trebuie să fie de tip ROM;
- ✓ La parcurgerea programelor de lucru, după execuția instrucțiunilor ce nu presupun efectuarea de salturi, actualizarea conținutului registrului PC se face plecând de la conținutul anterior la care se adaugă numărul de octeți pe care se reprezintă în memorie instrucțiunea executată; la procesoarele RISC acest număr rămâne întotdeauna constantă, la procesoarele CISC numărul depinde de tipul instrucțiunilor executate;
- ✓ În situația execuției instrucțiunilor ce presupun efectuarea de salturi, conținutul PC este încărcat cu adresa la care trebuie să se sară în programul de lucru; această adresă se încarcă direct în PC pentru situațiile de salt absolut, respectiv este făcută o sumă între PC-ul instrucțiunii care urmează după instrucțiunea curentă și distanța de salt (deplasament) pentru instrucțiunile de salt relativ;
- ✓ Pentru situația operării cu subrutine, conținutul PC-ului se salvează în memoria de tip stivă la apelul acesteia, iar adresei subrutinei se încarcă în PC. La sfârșitul execuției subrutinei conținutul anterior salvat în stivă se reîncarcă în PC continuându-se programul curent de lucru.

END 4 nov 2025

Semnificatia indicatorilor C(carry) si O(verflow)

C - are semnificatie pentru operatiile cu operanzi considerati de MPU fara semn (unsigned) si indica transportul (sau imprumutul) la operatiile de adunare (sau scadere) in raport cu bcms.

O – are semnificatie pentru operatiile cu operanzi considerati de MPU ca avand semn (signed – reprezentare in CC2) si indica depasirea gamei de reprezentare a rezultatului (in CC2 pe 8 biti, gama de reprezentare este [-128, 127]).

Nu exista o metoda prin care utilizatorul sa indice MPU cu ce operanzi va lucra (cu sau fara semn).

MPU proceseaza operanzii si ofera un acelasi rezultat, indiferent de tipul operanzilor, cu sau fara semn, impreuna cu informatii in registrul F (bitii C si O) care sa permita programatorului/compilerului sa interpreteze rezultatul obtinut (daca este corect sau nu).

Practic:

$$\begin{aligned} -x - (y) &= -x - y = \\ y - (-x) &= y + x \end{aligned}$$

$$x - y = x + (-y) \quad \text{scăderea se transformă în adunare}$$

(CC2)!

Dacă numerele care se aduna sunt de același semn și rezultatul este de semn opus, atunci apare depasire (O = 1) și rezultatul este incorect.

Dacă numerele care se adună sunt cu semne opuse, atunci nu apare niciodată depasire (O = 0).

~~Dacă se scade un număr pozitiv dintr-un număr negativ și rezultatul este pozitiv sau dacă un număr negativ este scăzut dintr-un număr pozitiv iar rezultatul este negativ, atunci apare o depasire (O = 1) și rezultatul este incorect.~~

Dacă numerele care se scad sunt de același semn, atunci nu se va produce niciodată o depasire (O = 0).

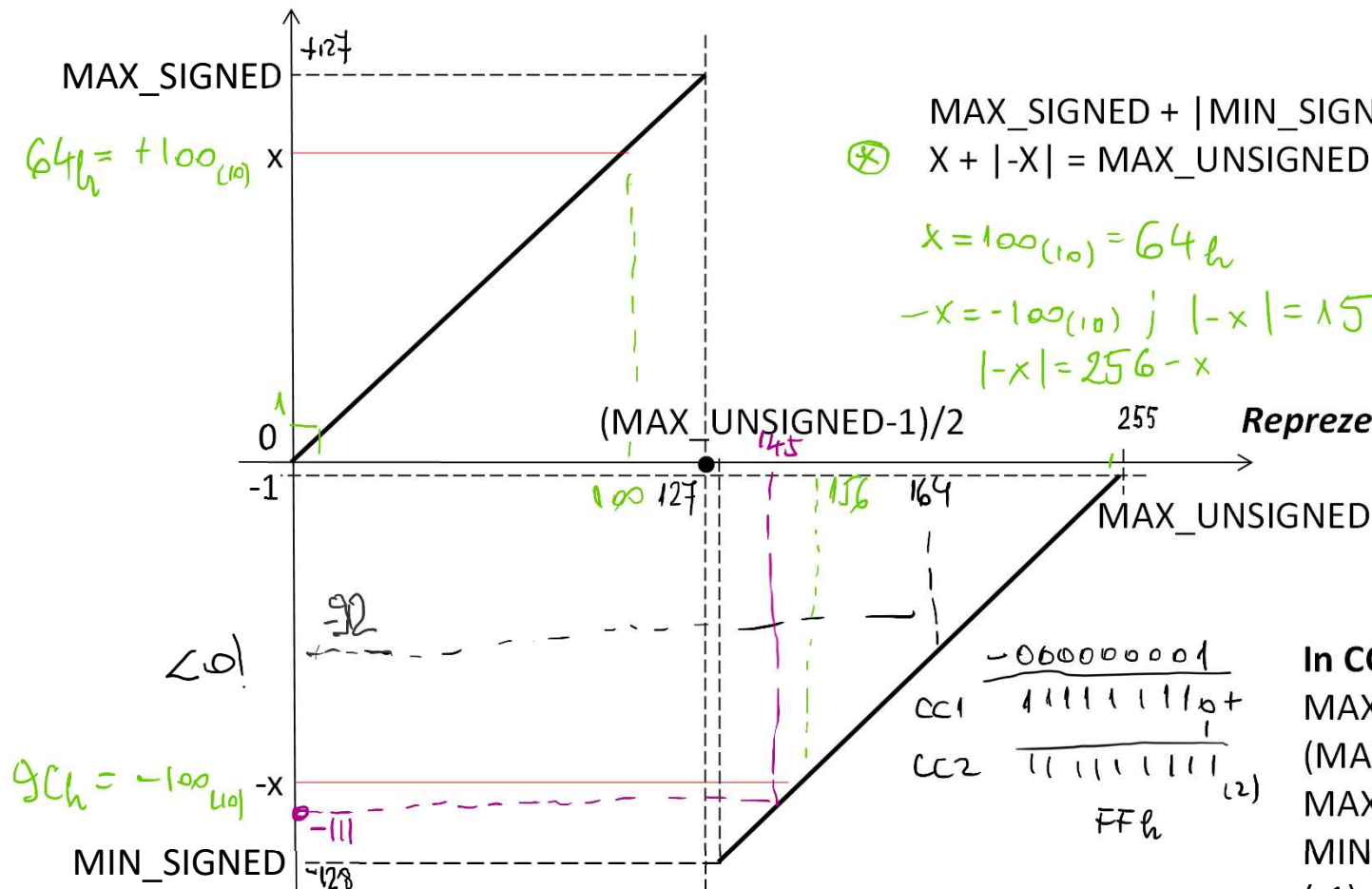
Ca metodă alternativă de detectare a depasirii pentru adunare, o depășire are loc dacă și numai dacă bitul de transport care intra in calculul bitului de semn (notat T_7) diferă de bitul de transport rezultat din calculul bitului de semn (care este notat C).

Adica: $O = C \oplus T_7$

Reprezentare cu semn vs. reprezentare fara semn

$$\begin{aligned}
 x &+ 1 & | -x | &= 256 - 1 = 255 = FFh \\
 -x &(-1) & &= FFh
 \end{aligned}$$

Reprezentare cu semn CC2



$$MAX_SIGNED + |MIN_SIGNED| = MAX_UNSIGNED$$

$$(*) \quad x + |-x| = MAX_UNSIGNED + 1$$

$$x = 100_{(10)} = 64h$$

$$-x = -100_{(10)}; \quad |-x| = 100_{(10)} = 64h = 1001100_{(2)} < 0$$

$$|-x| = 256 - x$$

Reprezentare fără semn

In CC2 pe 8 biti:

$$MAX_UNSIGNED = 255 = FFh$$

$$(MAX_UNSIGNED - 1)/2 = 127 = 7Fh$$

$$MAX_SIGNED = 127 = 7Fh$$

$$MIN_SIGNED = -128 = 80h$$

$$(-1)_{CC2} = FFh = 255$$

C – indica transport / imprumut la operatii aritmetice efectuate cu **operanzi fara semn**;
pe 8 biti, gama de reprezentare directa a numerelor fara semn este $[0, 255]$; pe 16 biti gama este $[0, 65535]$

O – indica depasirea gamei de reprezentare pentru rezultate obtinute la operatii aritmetice
efectuate cu **operanzi reprezentati cu semn** (CC2); pe 8 biti gama de reprezentare in CC2 este $[-128, 127]$

pe 16 biti in CC2 gama este $[-32768, +32767]$

Exemple de interpretare pentru indicatorii C(arry) si O(verflow) la reprezentarea pe 8 biti

Ex.1

$$104 + 45 = 68_{(16)} + 2D_{(16)} = 95_{(16)} = 149$$

$$104 < 127; 45 < 127; 149 < 255$$

Ambii operanzi au aceiasi reprezentare indiferent cum sunt considerati de MPU, ca avand semn sau nu.

Bitul C = 0 indica rezultat corect, daca operanzii sunt interpretati de MPU ca fiind fara semn!

Dar rezultatul $149 > 127 \Rightarrow O=1$!

Bitul O = 1 indica rezultat INCORECT IN CC2, daca operanzii sunt considerati cu semn de MPU!!!

Rezultatul incorect nu poate fi folosit pe mai departe in mod direct.

Observam ca $O = C \oplus T_7 = 0 \oplus 1 = 1$

O	C	T ₇							
1	0	1	1		1				
		0	1	1	0	1	0	0	0
		0	0	1	0	1	1	0	1
		1	0	0	1	0	1	0	1

68h
2Dh
95h

↑
m. negativ

Ex.2

$$104 + 145 = 68_{(16)} + 91_{(16)} = F9_{(16)} = 249$$

Bitul C = 0 indica rezultat corect daca operanzii sunt interpretati de MPU ca fiind fara semn!

Dar $145 > 127$! Deci 145 poate fi interpretat de MPU ca numar cu semn!!!

$$145 = -111_{(CC2)} \Rightarrow$$

Operatia de efectuat devine: $104 + (-111) = -7$ Rezultat considerat in CC2!

Reprezentarea in CC2 pentru -7 = $[!(0000\ 0111 - 1)] = 1111\ 1001_{(2)} = F9_{(CC2)}$ CORECT!!!

Bitul O = 0 indica rezultat corect in CC2! Observam ca $O = C \oplus T_7 = 0 \oplus 0 = 0$

O	C	T ₇							
0	0	0							
		0	1	1	0	1	0	0	0
		1	0	0	1	0	0	0	1
		1	1	1	1	1	0	0	1

68h
91h

$$145 + |-x| = 256$$

$$|-x| = 256 - 145 = 111$$

$$|-7| + x = 256 \quad x = 256 - 7 = 249 = F9h$$

Ex.3

$164 + 145 = A4_{(16)} + 91_{(16)} = 135_{(16)} = 309$

Bitul C = 1 indica rezultat incorect daca operanzii sunt interpretati de MPU ca fiind fara semn (309 > 255)!

Dar $164 > 127$! Deci 164 poate fi interpretat de MPU ca numar cu semn!!! $164 = -92_{(CC2)}$ ✓
Similar $145 > 127$! Deci 145 poate fi interpretat de MPU ca numar cu semn!!! $145 = -111_{(CC2)}$
Operatia de efectuat devine: $(-92) + (-111) = -203$ Rezultat considerat in CC2! Dar, $(-203) < (-128)$,
depasindu-se gama de reprezentare.

Bitul O = 1 indica rezultat incorect in CC2! Observam ca $O = C \oplus T_7 = 1 \oplus 0 = 1$

$164 + |-x| = 256 \Rightarrow |-x| = 256 - 164 = 92 \quad x = -92$
 $145 + |-x| = 256 \Rightarrow |-x| = 256 - 145 = 111 \quad x = -111$

O	C	T ₇							
1	1	0							
A4h		1	0	1	0	0	1	0	0
91h		1	0	0	1	0	0	0	1
		0	0	1	1	0	1	0	1

Ex.4

$244 + 19 = F4_{(16)} + 13_{(16)} = 107_{(16)} = 263$

Bitul C = 1 indica rezultat incorect daca operanzii sunt interpretati de MPU ca fiind fara semn (263 > 255)!

Dar $244 > 127$! Deci 244 poate fi interpretat de MPU ca numar cu semn!!! $244 = -12_{(CC2)}$
Operatia de efectuat devine: $(-12) + 19 = 7$ (Rezultat considerat in CC2).

Bitul O = 0 indica rezultat corect in CC2! Observam ca $O = C \oplus T_7 = 1 \oplus 1 = 0$

O	C	T ₇							
0	1	1	1	1					
F4h		1	1	1	1	0	1	0	0
13h		0	0	0	1	0	0	1	1
		0	0	0	0	0	1	1	1

END 11 nov 2025

Mecanismul de intreruperi

Întreruperile sunt evenimente speciale care apar pe parcursul rulării programului normal de lucru al unui procesor.

Întreruperile sunt de 2 tipuri: hardware, respectiv software.

Întreruperile hardware sunt evenimentele externe microprocesorului care sunt semnalizate acestuia pe pinii de intrare specializați: INT – interrupt și NMI – non maskable interrupt.

Intreruperile software reprezinta semnalizarea unor evenimente speciale petrecute în interiorul structurii sale (excepții), de exemplu semnalizarea operației de împărțire prin 0.

Întreruperile hardware sunt de două tipuri: mascabile și nemascabile. Denumirea se referă la posibilitatea de a le dezactiva pe cale software.

Întreruperile mascabile se pot dezactiva pe cale software, situație în care semnalizările nu au efect asupra procesorului. În situația în care sunt activate aceste tipuri de întreruperi semnalizările respective aplicate procesorului produc efect, procesorul trecând la tratarea respectivei întreruperi. Această categorie de întreruperi se aplică procesorului pe pini specializați cu notația INT. În general, numărul pinilor de tip INT este limitat (1 sau maxim 2), iar în situația în care numărul surselor de întreruperi este mare se utilizează circuite specializate de multiplexare a cererilor, circuite care se numesc Programmable Interrupt Controller (PIC).

Întreruperile nemascabile sunt acceptate în mod necondiționat de către procesor nefiind posibilă inhibarea lor software. Aceste cereri se aplică procesorului pe pini specializați cu denumirea NMI (non maskable interrupt). De obicei procesoarele au pin unic de tip NMI. Sursa unei cereri de întrerupere NMI este constituită de un eveniment cu prioritate maximală care poate să apară pe parcursul programului normal de lucru a procesorului.

Mecanismul de intreruperi

Întreruperile sunt evenimente speciale care apar pe parcursul rulării programului normal de lucru al unui procesor.

Întreruperile sunt de 2 tipuri: hardware, respectiv software.

Întreruperile hardware sunt evenimentele externe microprocesorului care sunt semnalizate acestuia pe pinii de intrare specializați: INT – interrupt și NMI – non maskable interrupt.

Intreruperile software reprezinta semnalizarea unor evenimente speciale petrecute în interiorul structurii sale (excepții), de exemplu semnalizarea operației de împărțire prin 0.

Întreruperile hardware sunt de două tipuri: mascabile și nemascabile. Denumirea se referă la posibilitatea de a le dezactiva pe cale software.

Întreruperile mascabile se pot dezactiva pe cale software, situație în care semnalizările nu au efect asupra procesorului. În situația în care sunt activate aceste tipuri de întreruperi semnalizările respective aplicate procesorului produc efect, procesorul trecând la tratarea respectivei întreruperi. Această categorie de întreruperi se aplică procesorului pe pini specializați cu notația INT. În general, numărul pinilor de tip INT este limitat (1 sau maxim 2), iar în situația în care numărul surselor de întreruperi este mare se utilizează circuite specializate de multiplexare a cererilor, circuite care se numesc Programmable Interrupt Controller (PIC).

Întreruperile nemascabile sunt acceptate în mod necondiționat de către procesor nefiind posibilă inhibarea lor software. Aceste cereri se aplică procesorului pe pini specializați cu denumirea NMI (non maskable interrupt). De obicei procesoarele au pin unic de tip NMI. Sursa unei cereri de întrerupere NMI este constituită de un eveniment cu prioritate maximală care poate să apară pe parcursul programului normal de lucru a procesorului.

La nivelul procesoarelor sunt diverse posibilități pentru tratarea întreruperilor apărute.

Obiectivul general al tehnicilor de tratare și al mecanismului hardware cu care un procesor este dotat în acest sens este acela de a direcționa procesorul către o subrutină specializată scrisă de către utilizator în vederea tratării întreruperilor apărute. Cum tratarea presupune apelarea unei subrutine specializate rezultă că urmează să fie folosit mecanismul de stivă și al registrului SP. Tratarea intreruperilor se poate face in 2 moduri: vectorizat si nevectorizat.

1. Pentru tratarea întreruperilor nemascabile, de obicei unice, procesorul este dotat cu o logică hardware care la apariția întreruperii apelează în mod necondiționat o subrutină de tratare a întreruperii specializată și aflată la o adresă fixă și rezervată.

Într-un sistem Z-80 ce utilizează întreruperi NMI utilizatorul este obligat ca în zona de memorie ce începe cu adresa 0066_H să depună un program ce tratează întreruperi nemascabile, natura programului este în funcție de aplicație.

Sfârșitul logic a unei subrutine de tratare a unei întrerupere nemascabile este tratat printr-o instrucțiune numită RETN (Return from NMI). Execuția acesteia conduce la reluarea activităților procesului din programul principal din locul suspendării ca urmare a cererii NMI.

2. Tratarea întreruperilor mascabile.

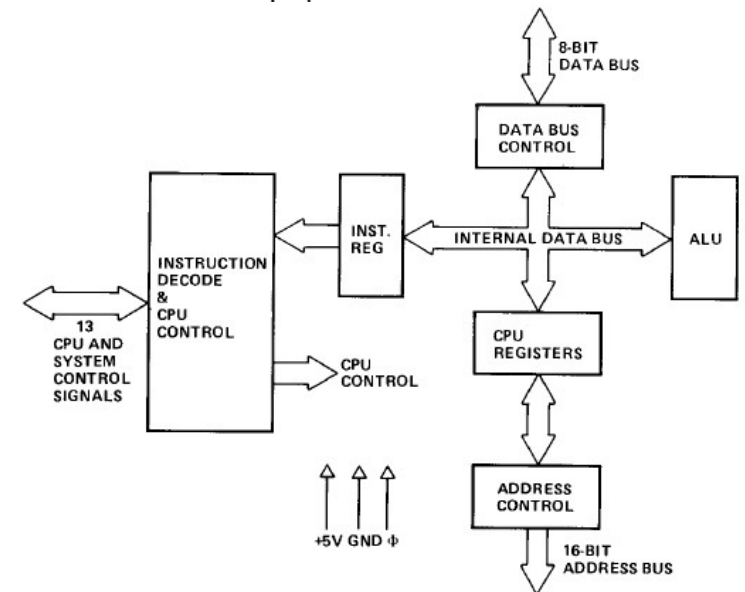
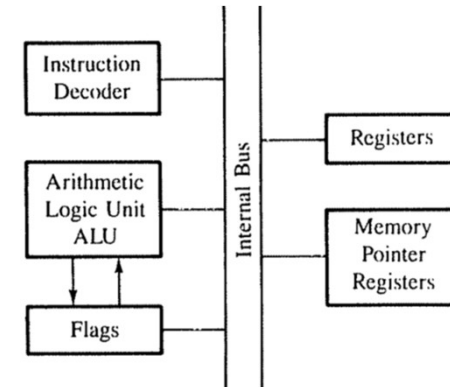
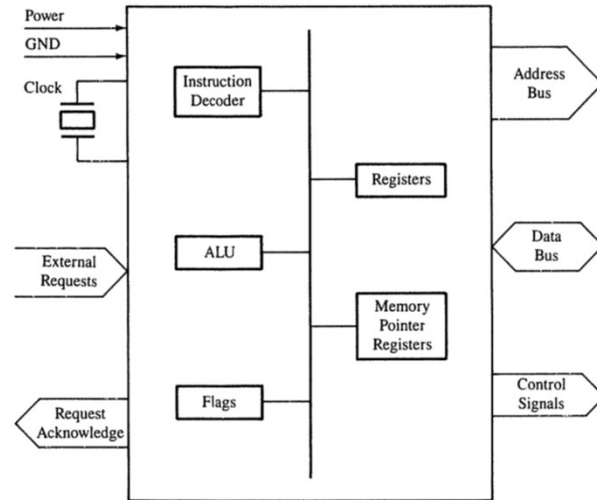
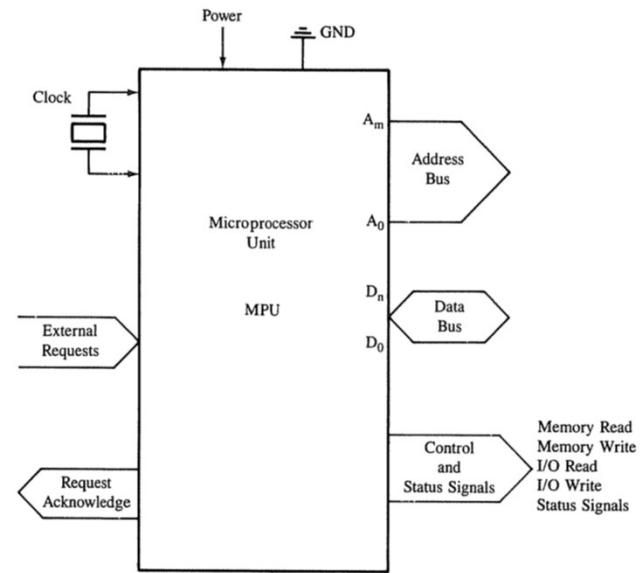
La nivelul procesoarelor care dispun de întreruperi mascabile există biți specializați accesibili software care activează/dezactivează mecanismul respectiv. De obicei starea mecanismului de întrerupere mascabilă este menținută de un bistabil denumit *I* care poate fi setat/resetat software.

În acest sens procesoarele dispun de diverse instrucțiuni: de ex. Z-80 – EI (Enable Interrupt) și DI (Disable Interrupt) sau X86 – STI (Set Interrupt) și CLI (Clear Interrupt). Când $I=1$, procesorul operează cu întreruperile mascabile iar când $I=0$ mecanismul este blocat.

Cel mai simplu mod de tratare apare și este utilizabil în sistemele de complexitate scăzută, în care există o singură sursă de întreruperi mascabile sunt. În acest mod de tratare mecanismele procesorului apelează în mod automat o subrutină pe care utilizatorul trebuie să o scrie la o adresă fixă și rezervată de memorie (la Z-80 adresa este 0038_H).

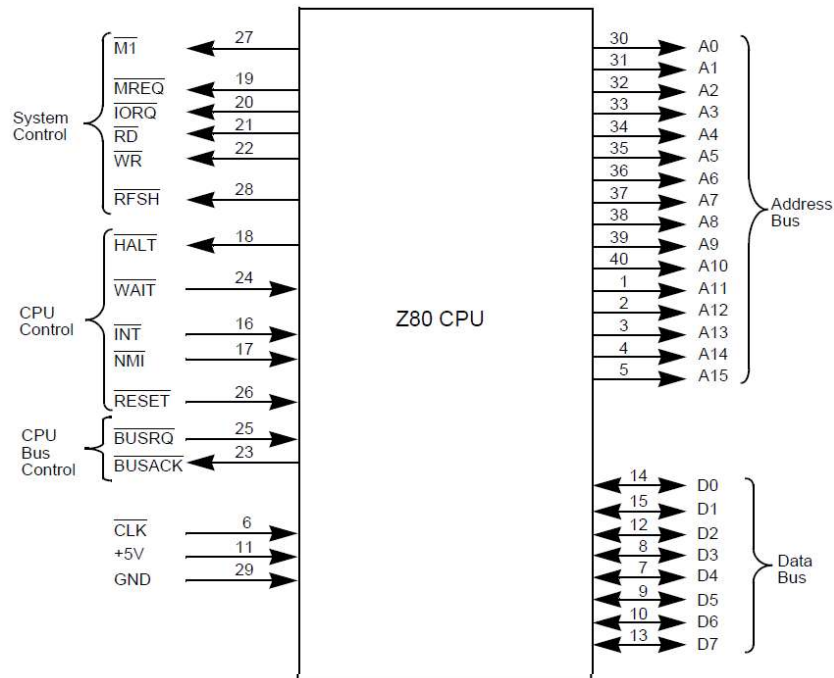
O subrutină de tratare a întreruperii mascabile se finalizează cu instrucțiunea RETI (Return from Interrupt).

MICROPROCESORUL Z80



MICROPROCESSORUL Z80

Z80 Hardware model



$\overline{M1}$ - Machine cycle one
 $\overline{MREQ}$ - memory request
 $\overline{IORQ}$ - i/o request
 $\overline{RFSH}$ - Refresh

Z80 Programming model

Main Register Set		Alternate Register Set	
Accumulator	Flags	Accumulator	Flags
A	F	A'	F'
B	C	B'	B'
D	E	D'	E'
H	L	H'	L'

General Purpose Registers

Interrupt Vector I	Memory Refresh R
Index Register	IX
Index Register	IY
Stack Pointer	SP
Program Counter	PC

Special Purpose Registers

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
S	Z		H		P/V	N	C

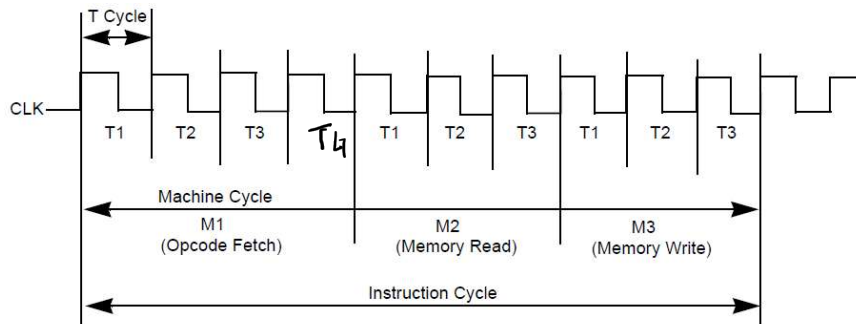
S = Sign
 Z = Zero
 H = Half-Carry
 P/V = Parity/Overflow
 N = Add/Subtract
 C = Carry

DIL - Dual In Line
 PLCC - Plastic Leaded
 Chip Carrier

END 18 nov 2025

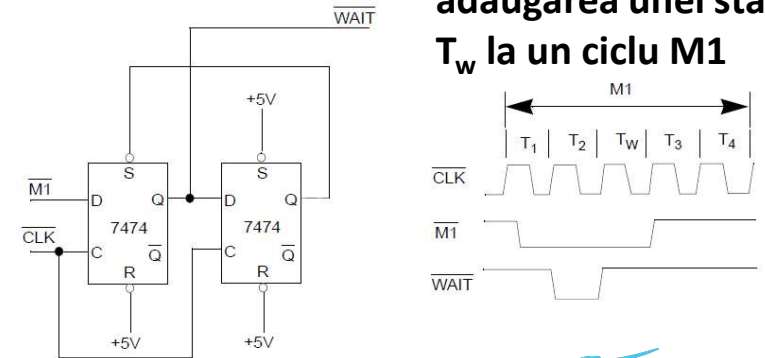
MICROPROCESORUL Z80

Z80 Ciclii masina si executia instructiunilor



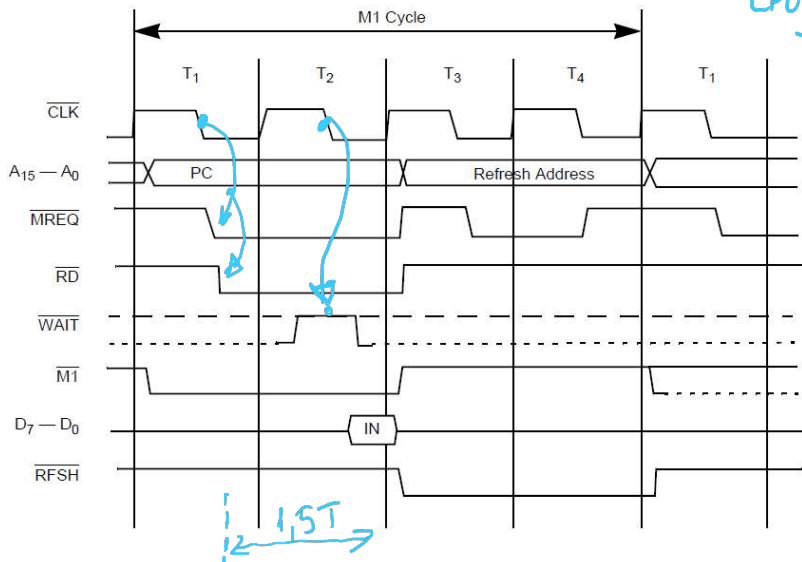
LD (HL), n8

Circuit pentru
adaugarea unei stari
 T_w la un ciclu M1



$$t_{a_{CPU}} = 3,5T$$

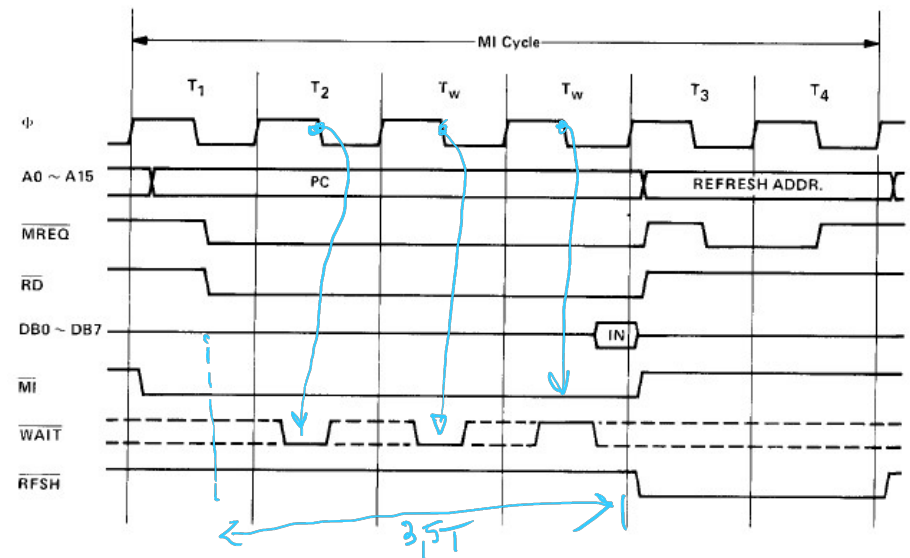
Fetch Cycle normal



$$t_{a_{CPU}} = 1,5T$$

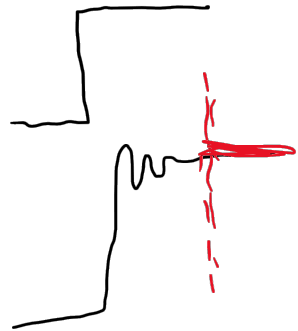
$$t_{MEM} \leq t_{a_{CPU}}$$

Fetch Cycle intarziat

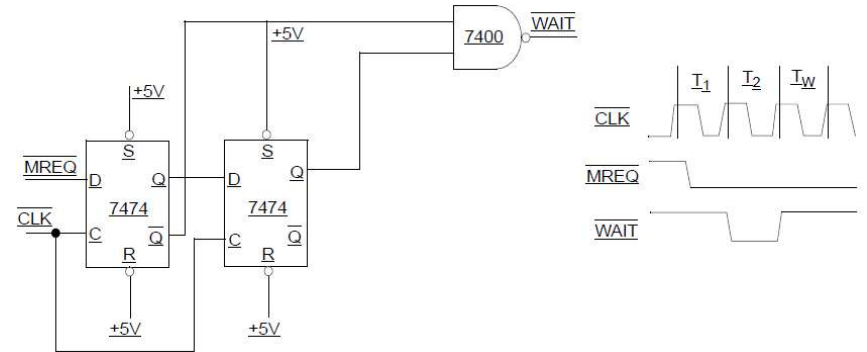


Z80 Cicli masina si executia instructiunilor

$t_{MEM} \leq t_{CPU}$!
 $t_{CPU} = 2T$

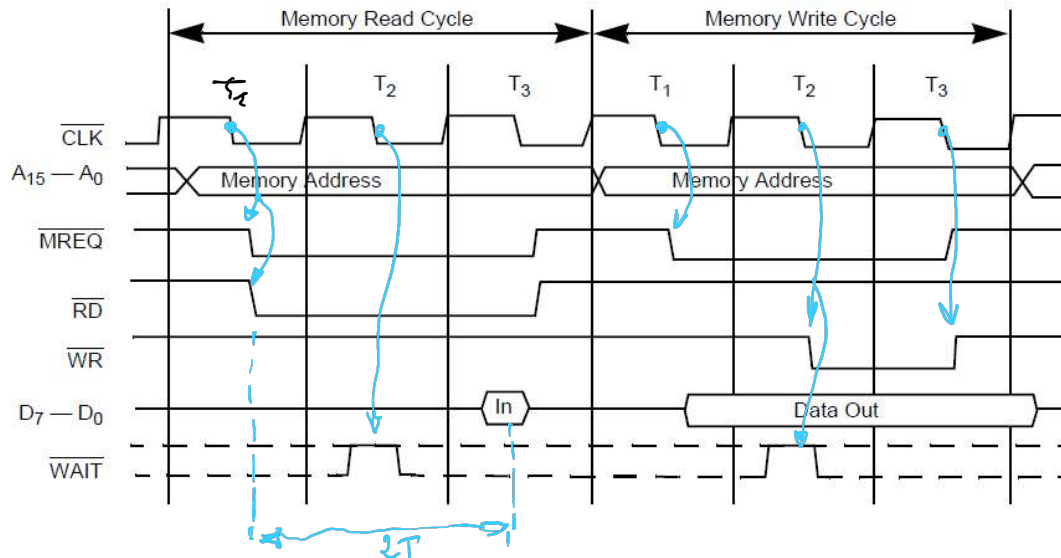


Circuit pentru adaugarea unei stari T_w la orice ciclu de lucru cu memoria



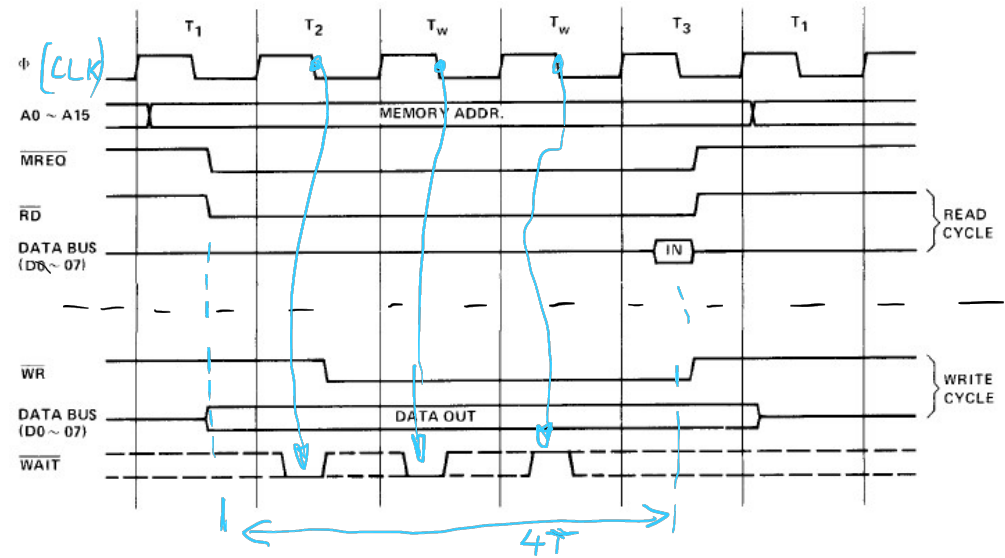
LD HL, n16

Memory Read Cycle and Memory Write Cycle - normal



Memory Read Cycle or Memory Write Cycle - intarziat

$t_{CPU} = 4T$



ex1 $LDA, n8 \quad A \leftarrow n8$
 1/- pe câți octeți se reprezintă?
 2/- cât durează executarea instr?

1/ $LDA, n8$
 1 octet 1 octet
 codul operand
 propriu-zis = 2 octeți

2/ Instrucțiunea este
 formată din
 - ciclu fetch $\rightarrow 4T$
 - ciclu citire operand $\rightarrow 3T$
 din memorie

ex4 LDA, B TOTAL $7T$
 1/ 1 octet
 2/ - ciclu fetch - $4T$

ex2 $LDA HL, n16 \quad HL \leftarrow n16$
 1/ 1 octet 2 octeți
 (2th) ($n16_L, n16_H$)
 ($n16_H, n16_L$)

TOTAL 3 octeți

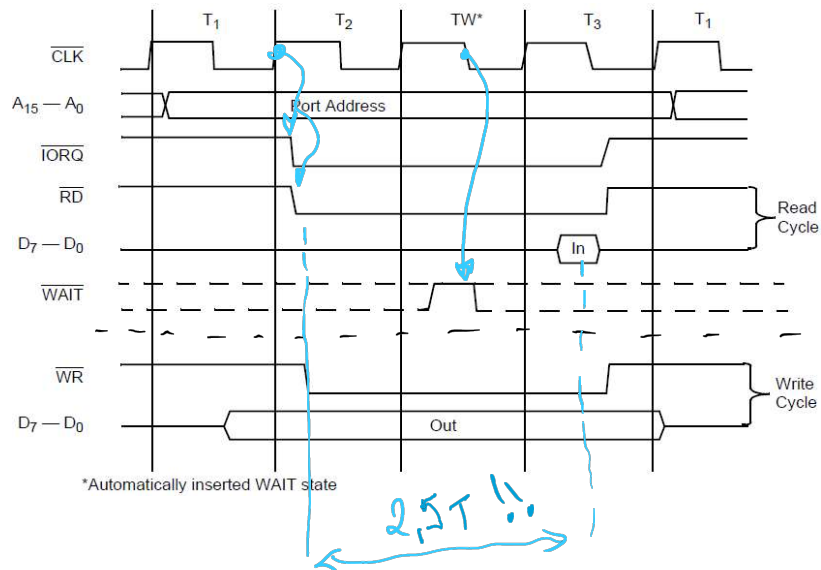
2/ - ciclu fetch $4T$
 - ciclu cit. OP.MEM $3T$
 - Ciclu cit OP.MEM $3T$
 $10T$

ex3
 1/ $LDA (n16), A \quad (n16) \leftarrow A$
 - codul propriu-zis - 1 octet!
 - operand $n16$ - 2 octeți
3 octeți
 2/ - ciclu fetch - $4T$
 - cit op mem ($n16_L$) - $3T$
 - cit op mem ($n16_H$) - $3T$
 - scr op mem - $3T$
 TOTAL $\rightarrow 13T$

Z80 Cicli masina si executia instructiunilor

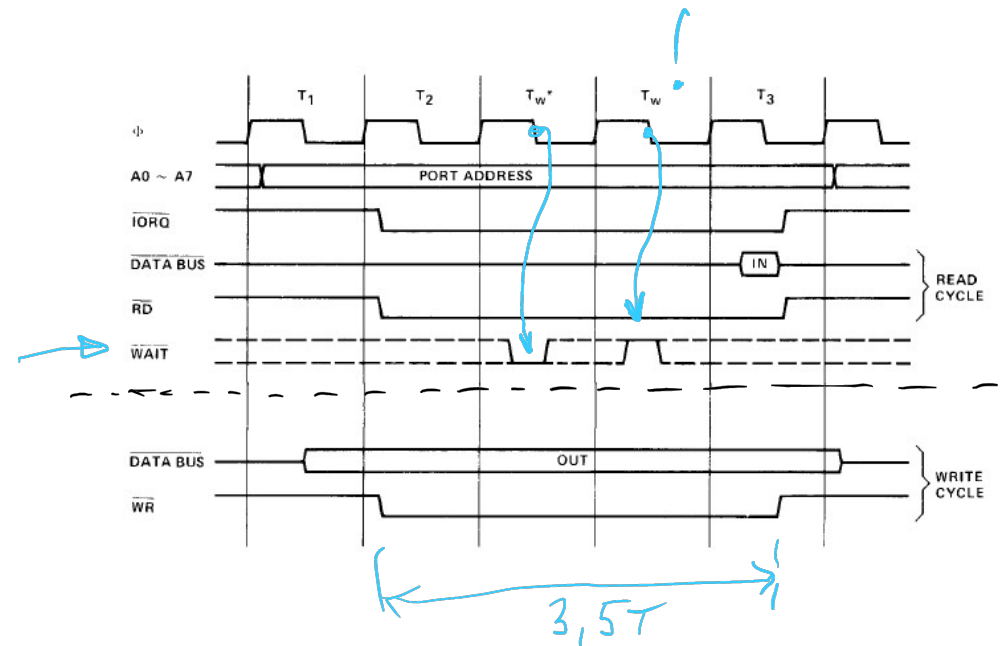
$$t_{P_{i/o}} \leq t_{a_{CPU}} = 2,5T$$

I/O Read Cycle or
I/O Write Cycle - normal



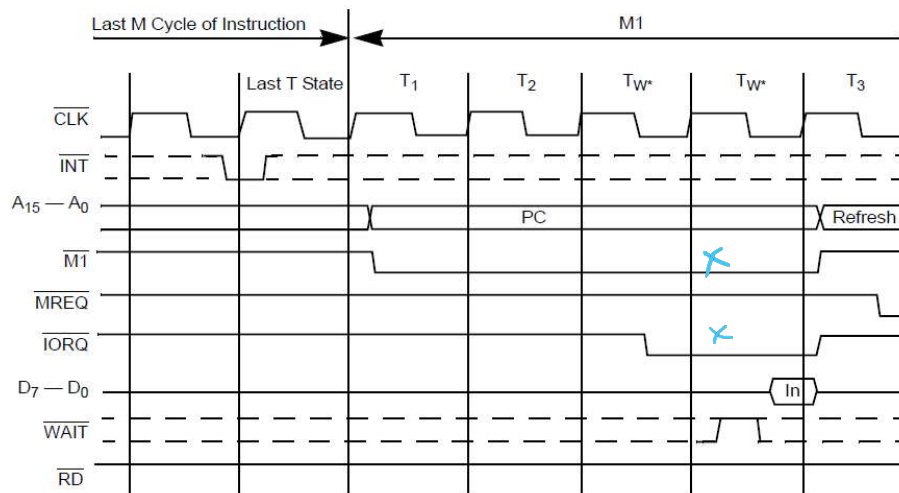
I/O Read Cycle or
I/O Write Cycle - intarziat

$$t_{a_{CPU}} = 3,5T$$



Z80 Cicli masina si executia instructiunilor

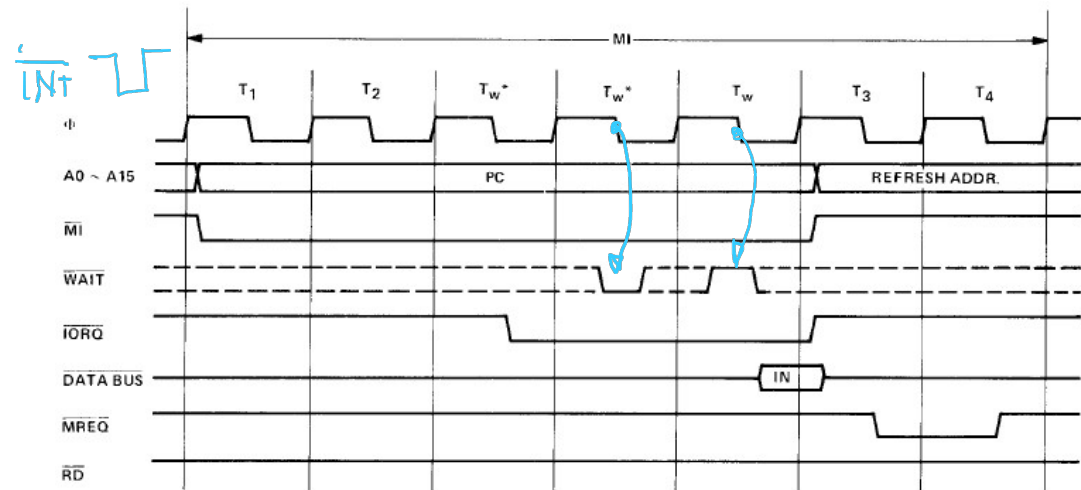
Interrupt Request/Acknowledge Cycle - normal



$\overline{M1} \wedge \overline{IORQ}$

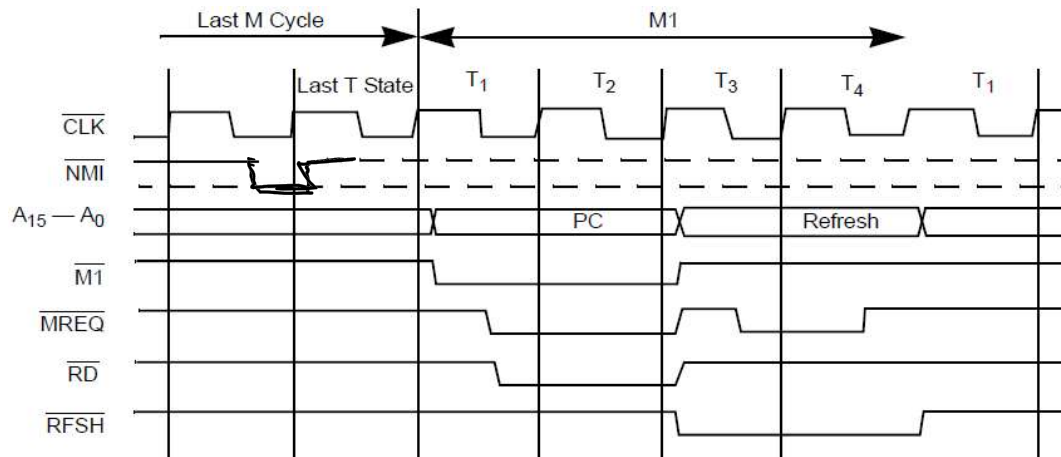
IN → info
pt that
intr. (subroutine)

Interrupt Request/Acknowledge Cycle - intarziat

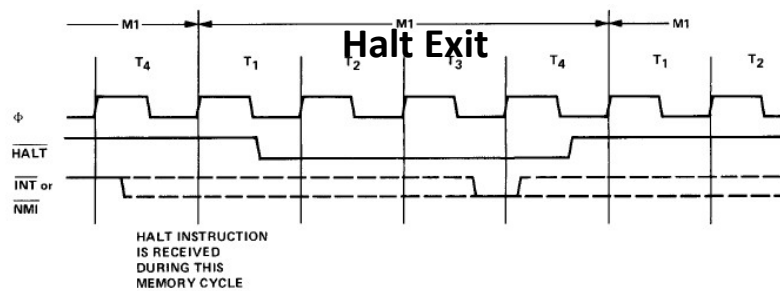
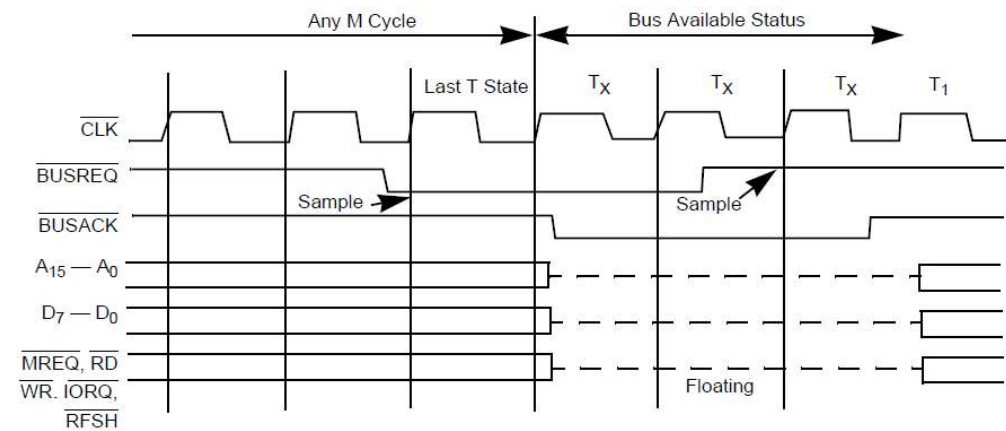


Z80 Ciclii masina si executia instructiunilor

Non Maskable Interrupt Cycle



Bus Request/Acknowledge Cycle

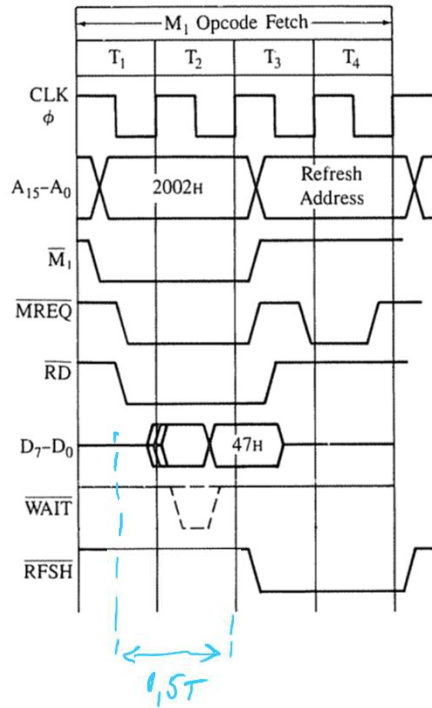


MICROPROCESORUL Z80

Z80 Ciclii masina si executia instructiunilor

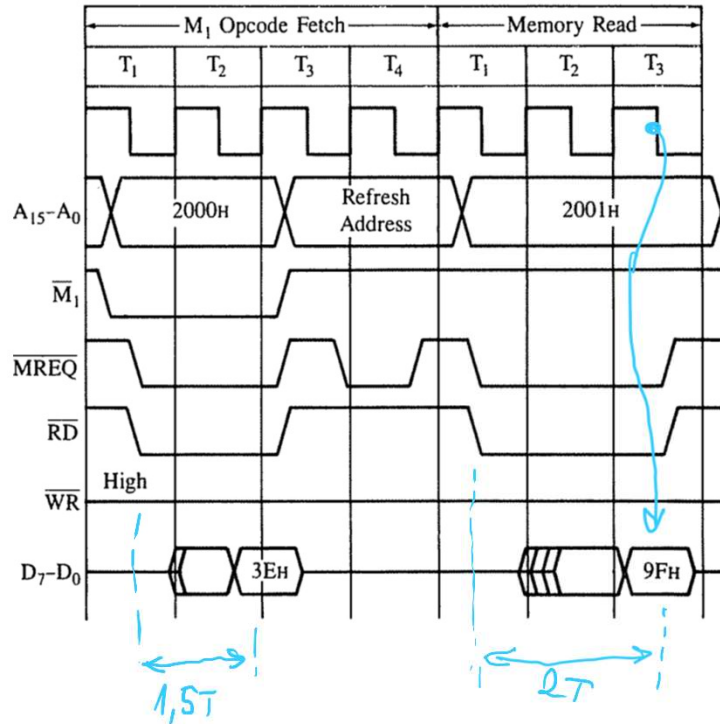
Fetch Cycle

2002h: LD B, A ; cod 47h



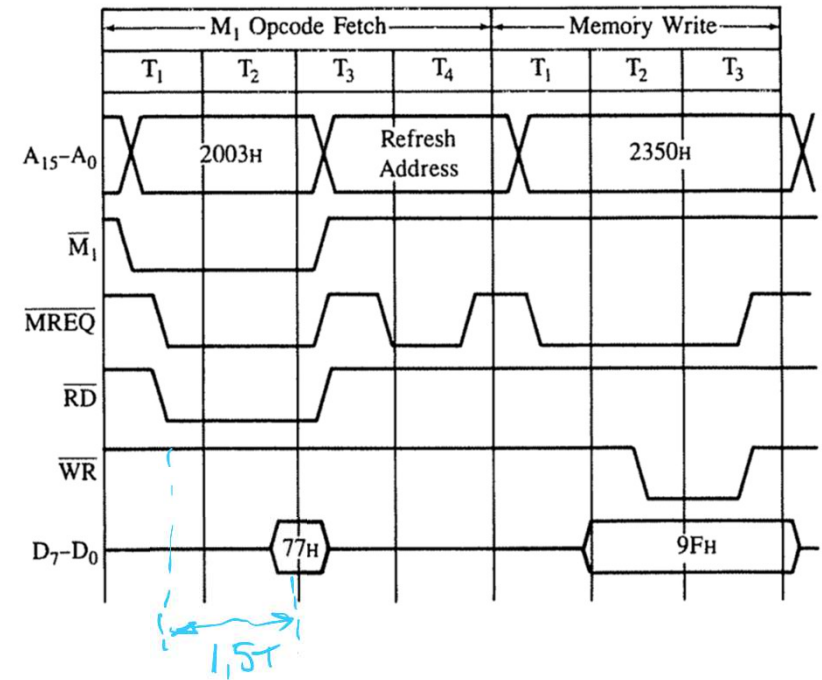
Memory Read Cycle

2000h: LD A, 9Fh ; cod 3Eh

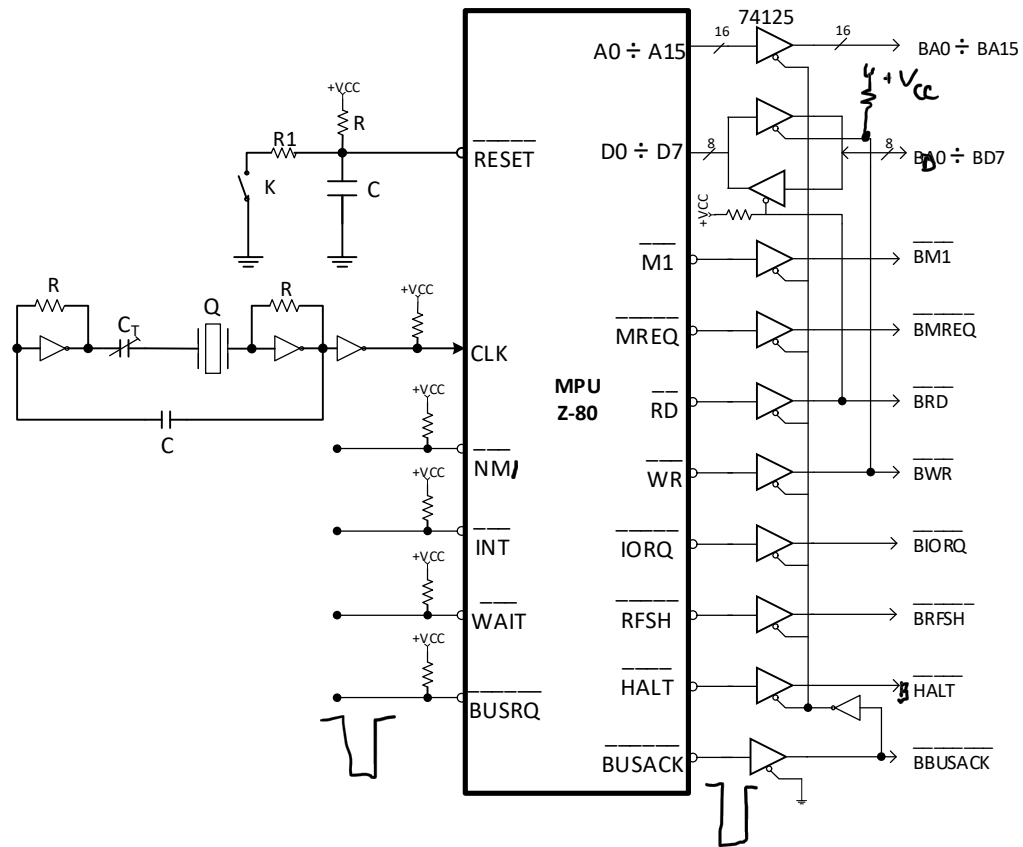


Memory Write Cycle

2003h: LD (HL), A; cod 77h, HL = 2350h, A=9Fh



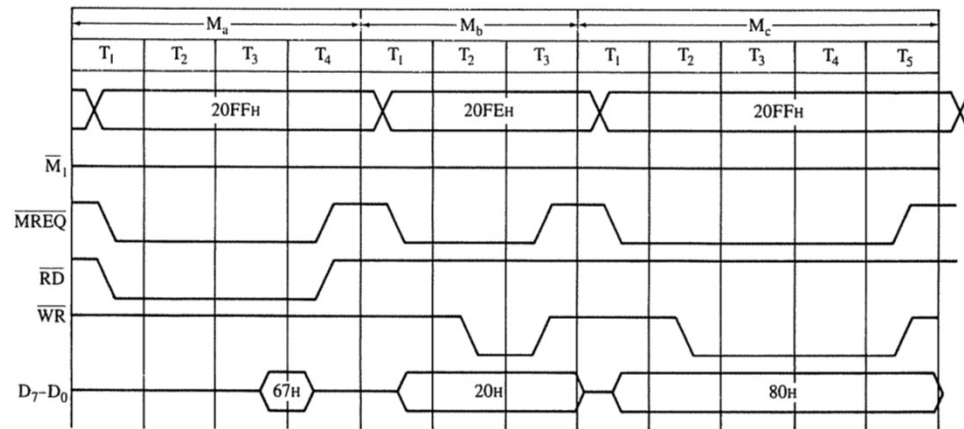
Interfatarea MPU Z-80



TEMA CURS

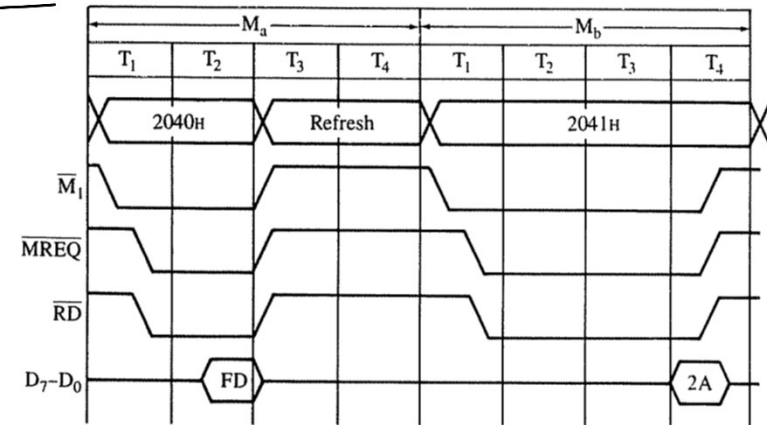
Analizati formele de unda din figurile de mai jos si raspundeti la intrebarile atasate:

P1



- Ce cicli masina executa Z-80 in conformitate cu formele de unda?
- Ciclii identificati pot face parte dintr-o instructiune? Daca da, care ar fi aceasta?

P2



Ce instructiune executa Z-80 in conformitate cu formele de unda?

25.Nov 2026

CONECTAREA CIRCUITELOR DE MEMORIE LA MAGISTRALELE MICROPROCESORULUI Z-80

Procesorul operează cu memoria prin utilizarea ciclurilor mașină de tip fetch, citire operand din memorie, respectiv scriere operand în memorie. Folosind semnalele din acești cicli, proiectantul unui microsistem de calcul urmează să genereze semnalele de comanda pentru circuitele de memorie în operațiile de accesare a acestora, folosind o logică digitală pe care trebuie să o proiecteze, respectând cerințele circuitului de memorie.

La conectarea circuitelor de memorie trebuie să se țină seama de următoarele reguli:

- ✓ La o adresă emisă de procesor trebuie să răspundă cel mult o locație de memorie; există posibilitatea ca o locație de memorie să răspundă la mai multe adrese emise de către procesor. O astfel de situație de adresare se numește adresare incompletă. Situația în care o locație de memorie răspunde exact la o adresă emisă de microprocesor se numește adresare completă. Adresarea incompletă poate fi folosită în sisteme de o complexitate redusă și conduce la un minim de hardware necesar.
- ✓ Respectarea încărcării maxime ale magistrelor sistemului de calcul; Circuitul de memorie se conectează la magistralele sistemului de calcul astfel încât să se respecte valorile maxime de încărcare a magistralei (atât din punct de vedere rezistiv cât și capacitiv).
- ✓ Pentru memoria de tip ROM proiectantul trebuie să asigure un mecanism de protecție pentru situațiile accidentale în care se forțează scrieri în acest tip de memorii.
- ✓ Verificarea încadrării timpului de răspuns al memoriei în timpul de așteptare al datelor de către procesor. Respectarea acestui principiu face ca microprocesorul să opereze la frecvențe maxime posibile. Dacă încadrarea menționată nu este realizată, proiectantul trebuie să acționeze asupra intrării *WAIT* în vederea trecerii procesorului în stări de așteptare până la primirea răspunsului memoriei. Timpul de răspuns al memoriei trebuie calculat luând în considerare momentul ultimei comenzi de accesare emisă de procesor pentru operarea cu memoria și până la primirea răspunsului așteptat de procesor (ținând seama de toate propagările prin circuite și magistrală).
- ✓ La accesarea memoriei RAM trebuie să fie premise atât operația de scriere cât și cea de citire. Suplimentar pentru memoria DRAM trebuie asigurată prin adresarea multiplexată accesarea, generarea semnalelor de control /RAS și /CAS, conectarea la magistrala de date funcție de tipul ciclului de scriere ales precum și operația de refresh a memoriei DRAM.

Problema 1. Să se conecteze la magistralele unui procesor Z-80, o zona de 24 Kocteti memorie ROM, folosind circuite de tipul 2764 cu organizarea 8K x 8. Adresa de început a zonei de memorie este 1000h, zona de memorie este continua, iar adresarea va fi completă.

Metodologie de rezolvare.

Etapa I.

1.1 Se prezinta schema circuitului concret de memorie cu care se face implementarea.

1.2 Se calculeaza numarul necesar de circuite de memorie si modul de organizarea a acestora in schema finala.

Etapa II.

2.1 Se intocmeste tabelul de adresare a zonei de memorie care va fi proiectata.

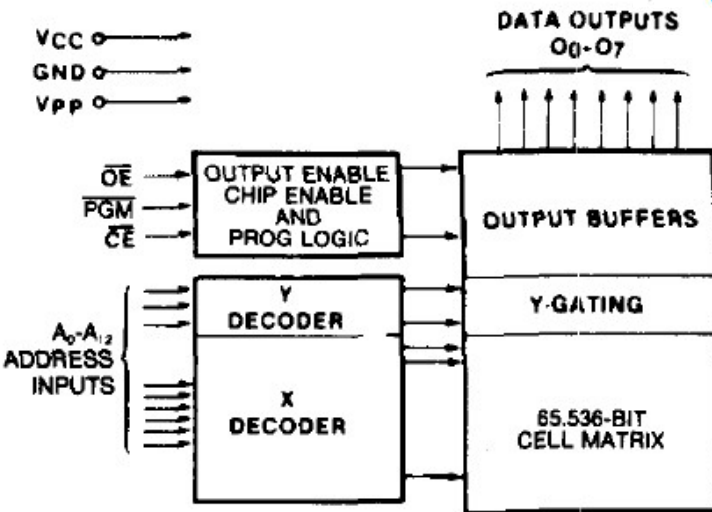
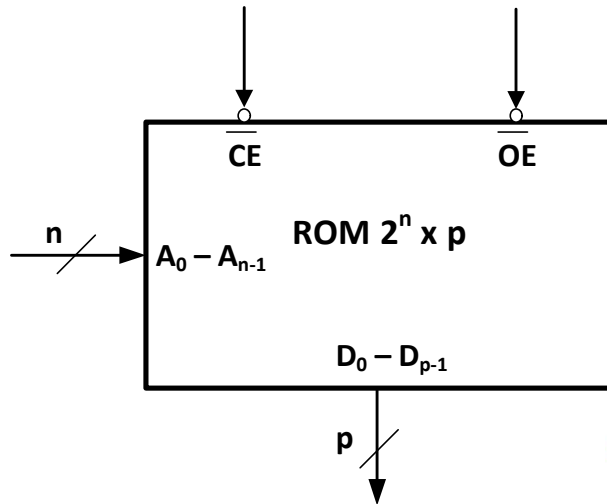
2.2 Se realizeaza schema bloc a zonei de memorie care va fi proiectata.

Etapa III. Implementarea propriu-zisa

3.1 Se proiecteaza logica de decodare si selectie a circuitelor de memorie din zona ceruta.

3.2 Se proiecteaza logica de conectare propriu-zisa a circuitelor de memorie din zona ceruta.

MEMORII ROM



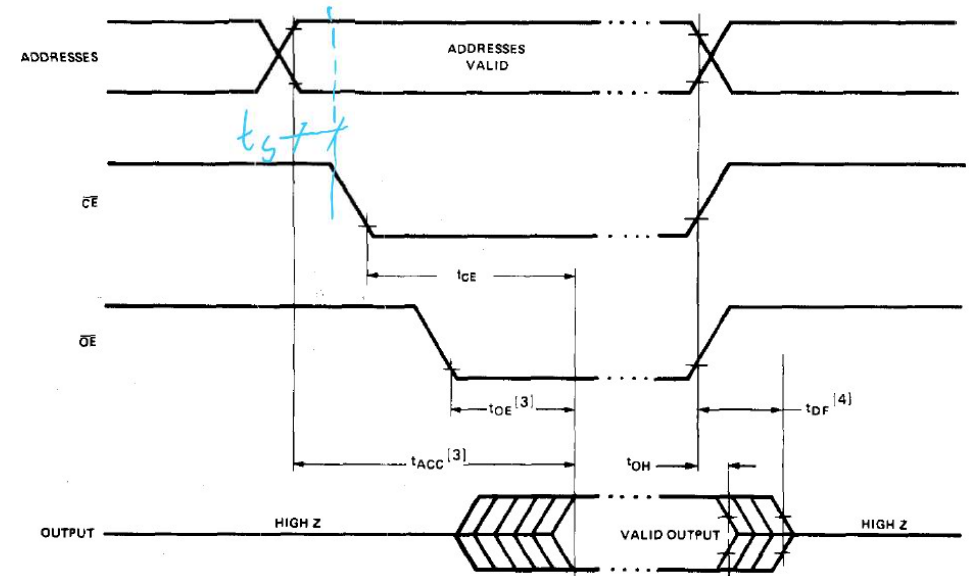
ROM
2764

8Kx8

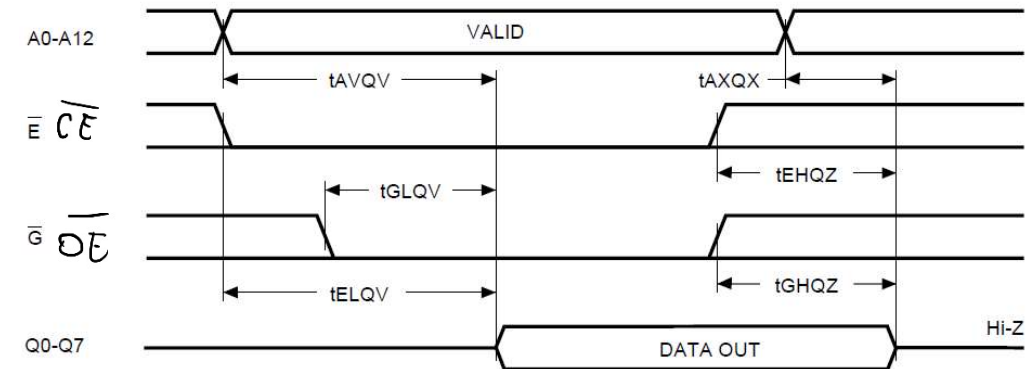
$\{ n=13$
 $p=8$

PIN NAMES

A_0-A_{12}	ADDRESSES
$\overline{CE}$	CHIP ENABLE
$\overline{OE}$	OUTPUT ENABLE
O_0-O_7	OUTPUTS
PGM	PROGRAM
N.C.	NO CONNECT

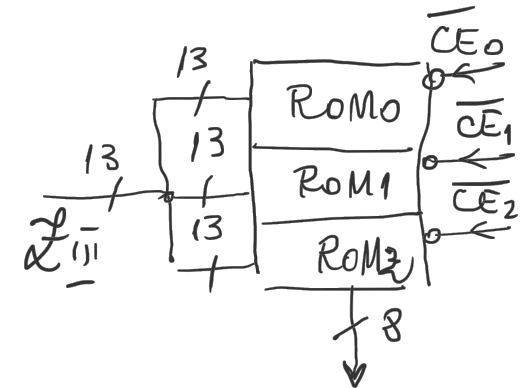
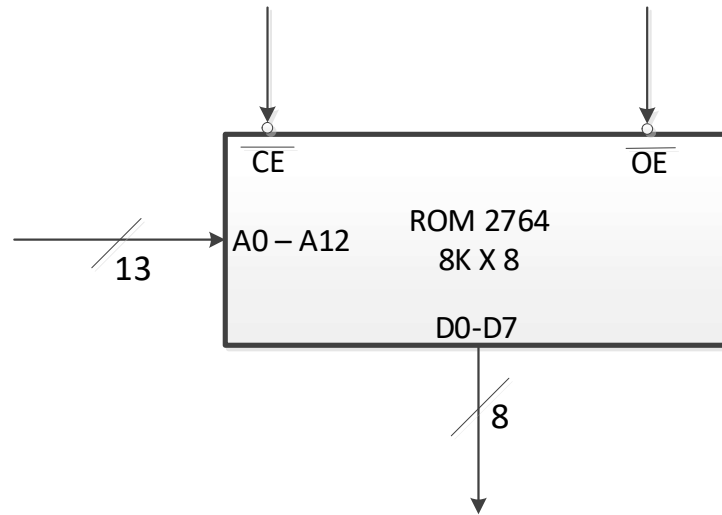


Read Mode AC Waveforms



Solutie

1.1 Schema circuitului concret de memorie cu care se face implementarea.



1.2 Calculul numarului necesar de circuite de memorie si modul de organizarea a acestora in schema finala.

$$\frac{\text{Organizare zona memorie}}{\text{Organizare circuit memorie}} = \frac{24k \times 8}{8k \times 8} = 3 \times 1 = 3 \text{ CI}$$

$\downarrow$
 linii x coloane

Rezulta ca este nevoie de 3 circuite de memorie ROM pe care le denumim **ROM0**, **ROM1** si **ROM2**, organizate pe 3 linii si o coloana.

2.1 Tabelul de adresare a zonei de memorie.

0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	← m. locatii memorie
A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	Adr. hex	Circuit
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1000h	ROM 0
0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	2FFFh	
0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	3000h	ROM 1
0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	4FFFh	
0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	5000h	ROM 2
0	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	6FFFh	

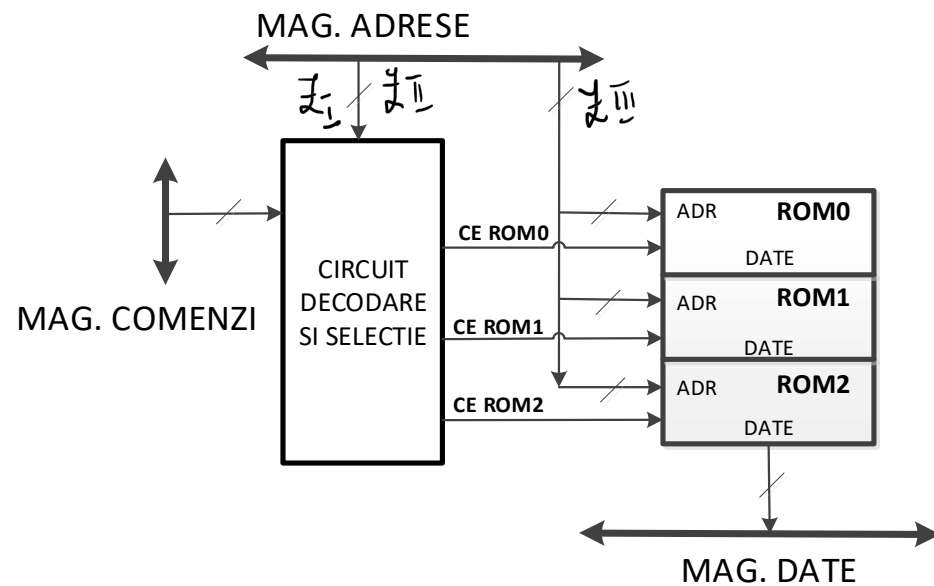
$\underbrace{\quad}_{Z_I}$
 $\underbrace{\quad}_{Z_{II}}$
 $\underbrace{\quad}_{Z_{III}}$

Zona I a tabelului cuprinde linii de adrese care raman nemodificate pe toata gama de adrese folosite pentru memorii.

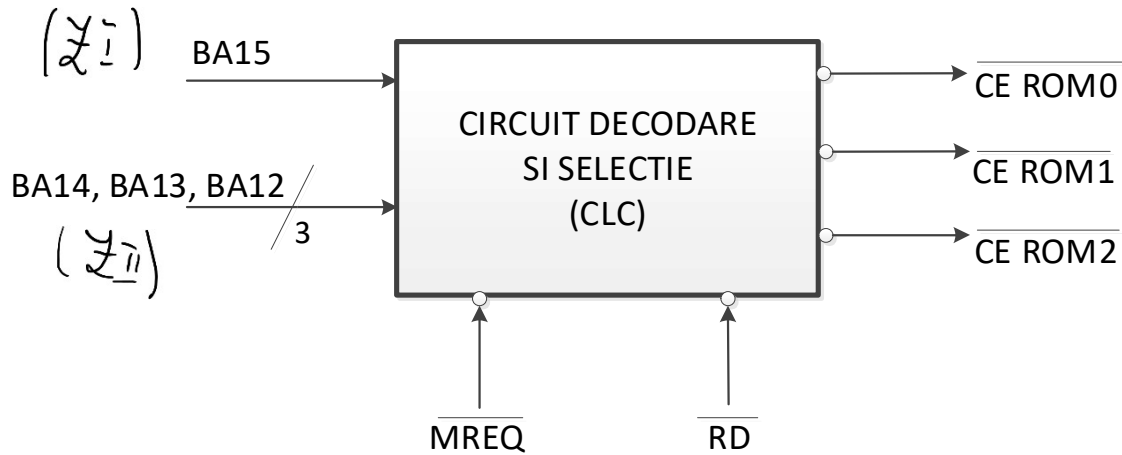
Zona II cuprinde liniile de adrese care permit selectia individuala a circuitelor de memorie folosite.

Zona III cuprinde liniile de adrese care se aplica tuturor circuitelor de memorie, iar numarul acestora este identic cu numarul intrarilor de adrese ale circuitelor respective.

2.2 Schema bloc a zonei de memorie care va fi proiectata.

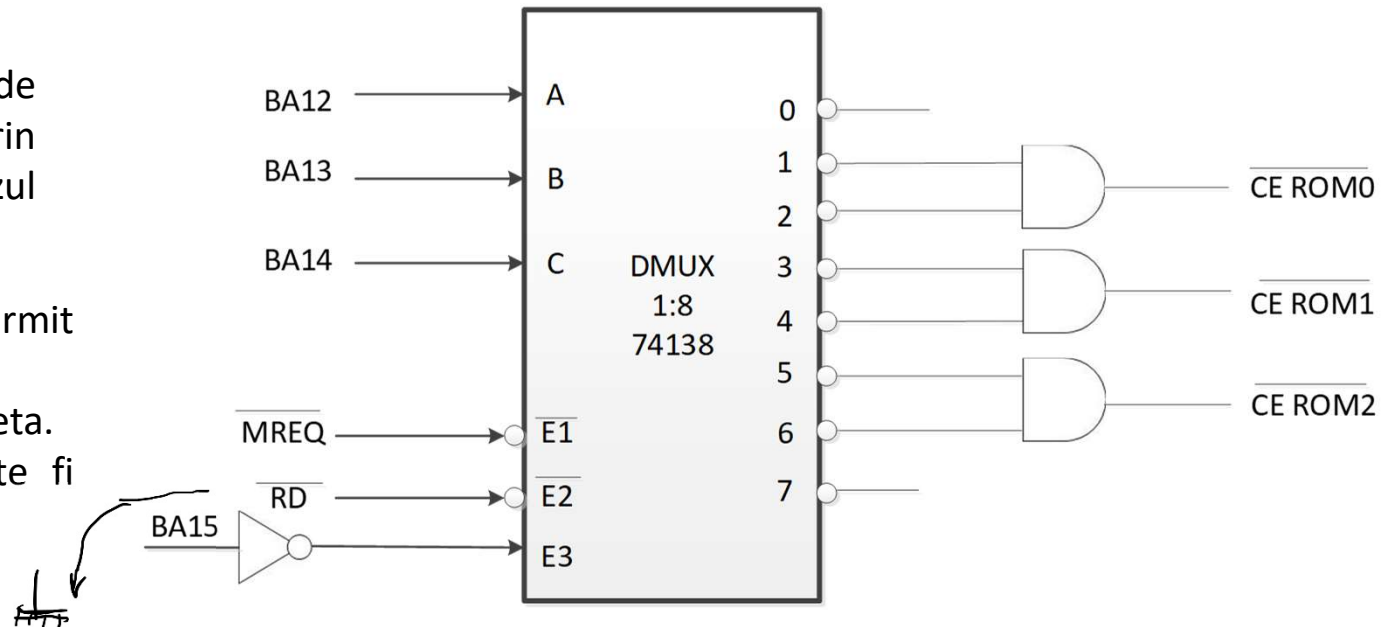


3.1 Proiectarea logicii de decodare si selectie a circuitelor de memorie din zona.

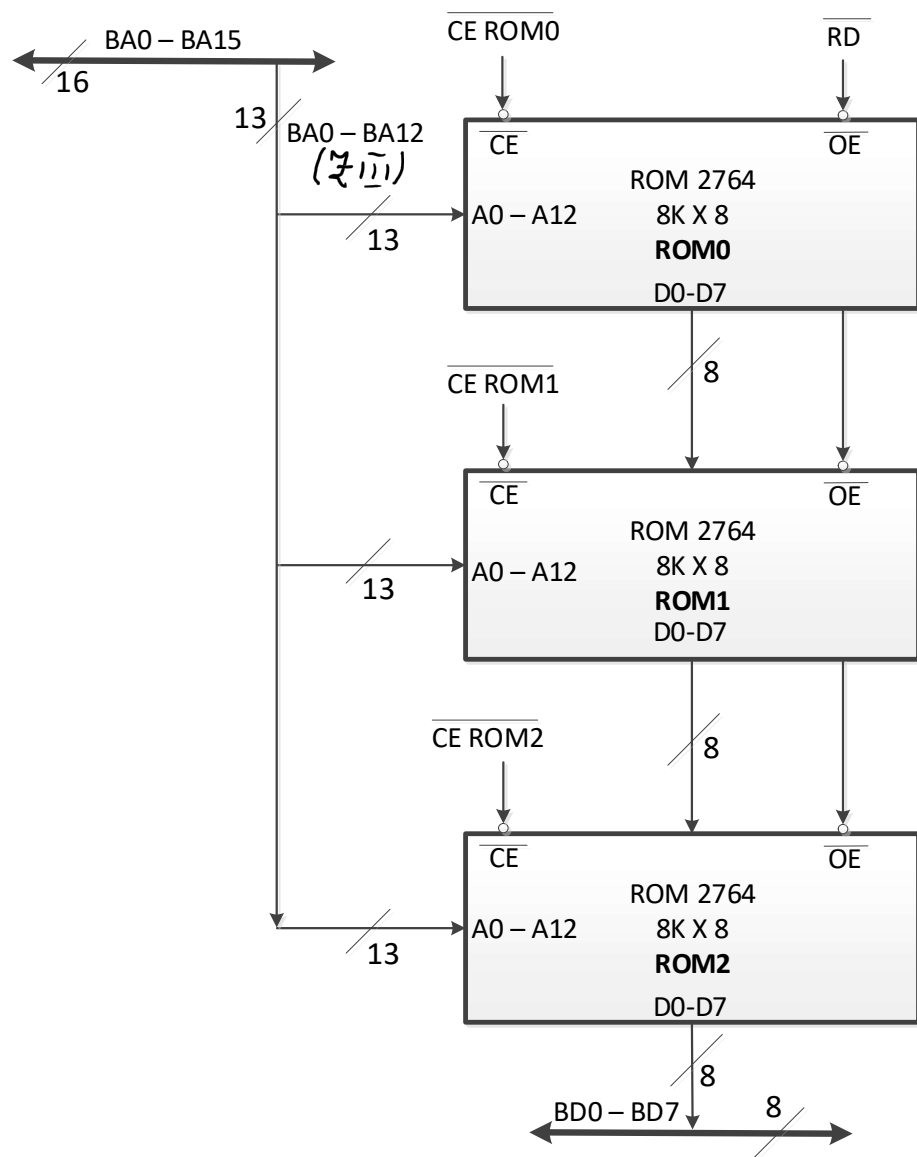


In schema bloc intervin semnalele generate de MPU care se activeaza in toti ciclii masina prin care procesorul opereaza cu memoria. In cazul nostru: fetch si citire operand din memorie.

Semnalele din zona II a tabelului de adrese permit selectia individuala a cicruitelor de memorie. Semnalele din zona I permit adresarea completa. Circuitul logic combinational – CLC – poate fi proiectat prin orice tehnica cunoscuta.



3.2 Proiectarea logicii de conectare propriu-zisa a circuitelor de memorie din zona ceruta.



Prin utilizarea semnalului /RD la intrarile /OE ale memoriilor se asigura protectia circuitelor ROM impotriva eventualelor scrieri accidentale si simultan deschiderea iesirilor acestora la operatiile de citire comandate de MPU.

Problema 2. Care este frecventa maxima a semnalului de ceas care se poate aplica procesorului, astfel incat la lucrul acestuia cu zona de memorie ROM proiectata sa nu fie necesare stari WAIT suplimentare? Se cunosc:

$$t_{p\text{ MAG DATE}} = 5ns; t_{p\text{ SSI}} = 10ns; t_{p\text{ MSI}} = 25ns; t_{A\text{ ROM}} = 150ns.$$

ssr - porti logice
msi - mux, dmux, add

Solutie. Este necesar sa fie verificata respectarea inegalitatii $t_{\text{asteptare MPU}} > t_{\text{raspuns memorie}}$ unde:

$t_{\text{asteptare MPU}}$ - este intervalul de timp in care MPU asteapta raspunsul memoriei, luand ca reper momentul lansarii ultimei comenzi care permite activarea memoriei si selectia locatiei cautate; acest timp este indicat de ciclul masina ai MPU prin care lucreaza cu memoria;

$t_{\text{raspuns memorie}}$ - este timpul care se scurge din momentul lansarii ultimei comenzi de catre MPU si pana cand datele din memorie ajung la MPU, luand in considerare toate magistralele si circuitele prin care se propaga informatiile; in acest timp se cumuleaza toti timpii necesari propagarii semnalelor, operarii corecte a circuitelor traversate precum si accesarii memoriei.

In cazul problemei de fata, MPU lucreaza cu memoria ROM folosind 2 tipuri de ciclul masina: fetch, respectiv citire operand din memorie. Cazul cel mai defavorabil este cel al ciclului fetch in care $t_{\text{asteptare MPU}} = 1,5 \cdot T$ unde T este perioada semnalului de ceas aplicata MPU.

Pentru calculul $t_{\text{raspuns memorie}}$ luam ca reper activarea semnalului MREQ, ultimul care conditioneaza activarea circuitului de memorie cautat (vezi formele de unda ale ciclului fetch) si parcurgem traseul acestuia sau ale semnalelor derivate de la MPU prin circuitul de decodare si selectie, prin memoria ROM si pe magistrala de date inapoi la MPU.

In cazul de fata se obtine:

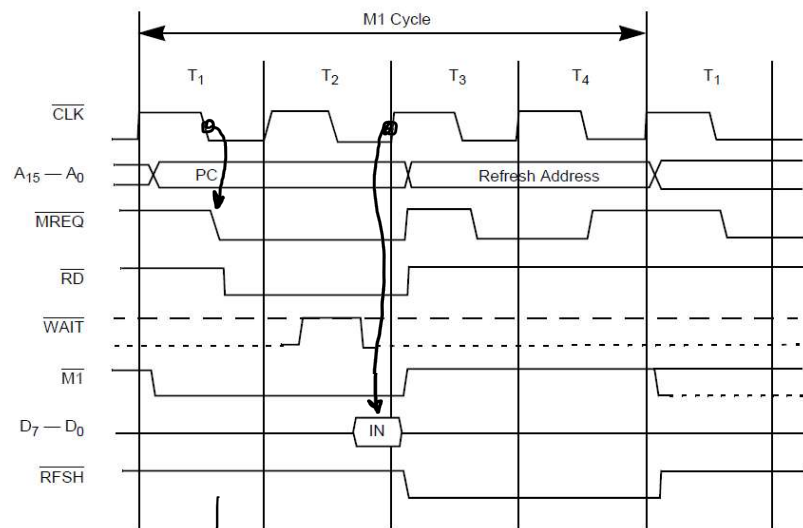
$$t_{\text{raspuns memorie}} = t_{p\text{ Buf}} + t_{p\text{ DMUX}} + t_{p\text{ SI}} + t_{A\text{ ROM}} + t_{p\text{ MAG DATE}} + t_{p\text{ Buf}} \text{ sau}$$

$$t_{\text{raspuns memorie}} = 10ns + 25ns + 10ns + 150ns + 5ns + 10ns = 210ns$$

Trebuie sa avem: $t_{\text{asteptare MCU}} = 1,5T > t_{\text{raspuns memorie}} = 210ns$ sau $T = 1/f > 210ns \cdot 2/3 = 420ns/3 = 140ns$

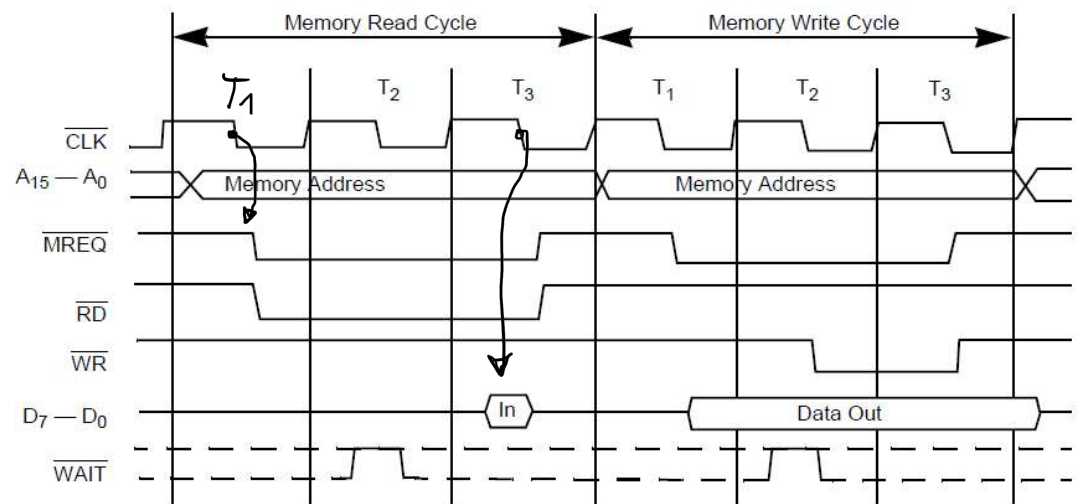
De unde $f < \frac{1}{0,140} MHz = 7,143 MHz$ - frecventa maxima ceruta in enunt.

Fetch Cycle normal



$t_{w\text{ MPU}} = 1.5 T$
caz defazível

Memory Read Cycle and Memory Write Cycle - normal



$t_{w\text{ MPU}} = 2 T$

Problema 3. In zona de memorie ROM proiectata, la locatia cu adresa $12345_{(10)}$ continutul programat a fost $167_{(10)}$. Efectuand o citire de la aceiasi adresa, MPU regaseste data $231_{(10)}$. Care este circuitul ROM posibil defect si pe ce linii de date apare anomalia semnalata?

Solutie.

Locatia cu adresa $12345_{(10)}$ este de fapt $3039h$. In tabelul de adrese realizat anterior, constatam ca locatia respectiva se afla in circuitul ROM1 (posibil defect).

Din punct de vedere al datelor avem:

$167_{(10)} = A7h = 1 \underline{0} 1 0 0 1 1 1_{(2)}$
 $231_{(10)} = E7h = 1 \underline{1} 1 0 0 1 1 1_{(2)}$

Se constata ca linia de date care trebuie investigata este D6 (liniile de date fiind notate D7 – D0).

END 2 dec 2025

